JukeBox GR4 – Opgaveplan

Mini-projekt · Prototype 1 · IT-arkitektur, 1. semester

Målet: En simpel jukebox i browseren, hvor man kan

- klikke på **Albums** og folde hvert album ud, så man ser albummets sange,
- klikke på **Kunstner** og folde hver kunstner ud, så man ser kunstnerens albums,
- trykke **"Tilføj til køen"** ved en sang, så den lander i en **kø-boks**.

Alt skal være så simpelt som muligt. Vi bruger **Bootstrap Accordion** til fold-ud-menuerne.

Det har vi allerede

Del	Hvad den kan
Layout	Topbar og tre kolonner med flexbox. Midterkolonnen kan scrolles (`overflow-y: auto`).
`hentArtists()`	Henter `artists.json` og laver en knap pr. kunstner.
`hentSange()`	Henter sange *og* kunstnere og viser "Sang – Kunstner" (opslag på tværs af filer).
`hentAlbums()`	Samme princip med "Album – Kunstner".

Vi kan mønstret **fetch** → **array** → **løkke** → **createElement** → **appendChild**. Alt det nye bygger videre på det.

Indhold

1. **Del A – Omlægning af filer** (laves af én person, før alt andet)
2. **Del B – Opgaver** (kø, sang-linje, accordion, albums, kunstnere)
3. **Del C – Fordeling, fejlfinding og links**

Del A – Omlægning af filer

> **Vigtigt:** Del A laves af **én person**, mens de andre venter. Hvis flere flytter rundt på filer samtidig, får vi garanteret Git-konflikter. Når Del A er færdig: **commit + push**, og så laver alle andre `git pull`.

Hvorfor dele op i flere filer?

- **Færre Git-konflikter:** Vores sidste merge-konflikt skete, fordi alle arbejdede i `W3S_1.html`. Med én fil pr. gruppe redigerer vi sjældent samme sted.
- **Overblik:** Hver fil har ét ansvar, så det er nemt at finde ting.
- **God praksis:** HTML = struktur, CSS = udseende, JS = opførsel ("separation of concerns").

Den nye struktur

```
Prototype_1/
├── db/
│   ├── albums.json
│   ├── artists.json
│   └── sange.json
└── Frontend/
    ├── index.html           ← forsiden (link rettes til jukebox.html)
    ├── styles.css           ← forsidens stil (røres ikke)
    ├── jukebox.html         ← NY: tidligere W3S_1.html, nu KUN HTML
    ├── css/
    │   └── jukebox.css      ← NY: jukeboxens egen stil
    └── js/
        ├── faelles.js       ← NY: hjælpefunktioner som alle bruger
        ├── koe.js           ← NY: køen
        ├── sange.js         ← NY: hentSange()     (Shanjeef)
        ├── albums.js        ← NY: hentAlbums()    (Abi & Phillip)
        └── artists.js       ← NY: hentArtists()   (Simon & Oliver)
```

Trin A1 – Opret mapper

Højreklik på `Frontend` i VS Code → **New Folder** → lav `css` og `js`.

Trin A2 – Omdøb HTML-filen

Højreklik på `W3S_1.html` → **Rename** → `jukebox.html`. Ret derefter linket i `index.html`:

```
<a href="jukebox.html">Jukebox</a>
```

Trin A3 – Flyt CSS til css/jukebox.css

1. Opret filen `css/jukebox.css`.
2. Klip **alt indholdet** mellem `<style>` og `</style>` fra `jukebox.html` og sæt det ind i `jukebox.css` (uden selve style-tags).
3. Kopiér de fire accordion-regler (`.accordion-button`, `.accordion-button:hover`, `.accordion-button:not(.collapsed)`, `.accordion-body`) fra `styles.css` ind nederst i `jukebox.css`.
4. Slet den nu tomme `<style></style>`-blok i `jukebox.html`.

Trin A4 – Flyt JavaScript til js/

Fil	Hvad flyttes derhen
`js/sange.js`	Hele funktionen `hentSange()`
`js/albums.js`	Hele funktionen `hentAlbums()`
`js/artists.js`	Hele funktionen `hentArtists()` + linjen `hentArtists();` nederst (hvis vi vil beholde, at kunstnerne vises fra start)
`js/faelles.js`	Tom for nu – bare en kommentar: `// Fælles hjælpefunktioner`
`js/koe.js`	Tom for nu – bare en kommentar: `// Køen`

Derefter slettes hele `<script> ... </script>`-blokken i `jukebox.html` – **også det ekstra `</script>`** der stod nederst.

Trin A5 – Kæd filerne sammen i jukebox.html

Sådan skal `jukebox.html` se ud bagefter. Bemærk: `class="kolonne accordion"` på `#liste` er nyt – den skal bruges af Bootstrap senere.

```
<!DOCTYPE html>
<html lang="da">
<head>
  <meta charset="UTF-8">
  <title>JukeBox GR4</title>

  <!-- 1. Bootstrap CSS (hentes fra nettet) -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <!-- 2. Vores egen CSS – EFTER Bootstrap, så vores regler vinder -->
  <link href="css/jukebox.css" rel="stylesheet">
</head>
<body>

<div class="bar">
  <div class="barEmner">JukeBox GR4</div>
  <div class="barSearch" style="justify-content:center">
    <input>
  </div>
  <div class="barEmner">Kreditter</div>
</div>

<div class="alleKolonner">
  <div class="kolonne" style="flex:0.25; justify-content:center">
    <button onclick="hentAlbums()">Albums</button>
    <button onclick="hentArtists()">Kunstner</button>
    <button onclick="hentSange()">Sange</button>
  </div>

  <div id="liste" class="kolonne accordion" style="padding-top:30px">
  </div>

  <div class="kolonne">
    <div style="height:100px; border: 3px solid green"></div>
  </div>
</div>

<!-- 3. Bootstrap JS – får accordion til at folde ud -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>

<!-- 4. Vores egne filer – rækkefølgen betyder noget! -->
<script src="js/faelles.js"></script>
<script src="js/koe.js"></script>
<script src="js/sange.js"></script>
<script src="js/albums.js"></script>
<script src="js/artists.js"></script>

</body>
</html>
```

Hvorfor kan albums.js bruge en funktion fra faelles.js?
Almindelige `<script>`-filer deler **ét fælles rum** (det globale scope). Browseren læser filerne oppefra og ned, og alt der står "øverst" i filerne (funktioner, `let koe = []`) havner i samme rum. For browseren er det som én lang fil – vi har bare delt den op for vores egen skyld.

Rækkefølgen er vigtig for kode der kører *med det samme* (fx `hentArtists();`). Derfor står fælles-filerne først. Funktioner der kaldes ved et klik er ikke så sarte – når man klikker, er alle filer for længst indlæst.

Faldgrube – stier i fetch: En fetch-sti læses ud fra **HTML-filen**, ikke JS-filen. Selvom koden nu ligger i `js/albums.js`, er stien stadig `"../db/albums.json"` (set fra `Frontend/jukebox.html`). Altså: **stierne skal ikke ændres**.

Bemærk: Bootstrap ændrer standardudseendet lidt (knapper, skrifttype, marginer). Det er ikke en fejl – siden kan bare se en smule anderledes ud end før.

Trin A6 – Test at alt virker som før

Åbn `jukebox.html` med **Live Server**, åbn konsollen (F12 → Console) og tjek:

- ☐ Ingen røde fejl i konsollen
- ☐ "Albums" viser albums
- ☐ "Kunstner" viser kunstnere
- ☐ "Sange" viser sange
- ☐ Den grønne layout-ramme ser ud som før (CSS'en er fundet)

Trin A7 – Del det med gruppen

```
git add .
git commit -m "Omlægning: opdelt i css/ og js/, omdøbt til jukebox.html"
git push
```

Derefter laver **alle andre**: `git pull`

📖 Læs: w3schools – JS Where To (External JavaScript) · HTML Styles – External CSS · HTML File Paths · JS Scope

Del B – Opgaver

Overblik: sådan hænger det sammen

```
[Albums]   → hentAlbums()  → accordion pr. album    → sange i albummet ─┐
[Kunstner] → hentArtists() → accordion pr. kunstner → albums            │
[Sange]    → hentSange()   → alle sange ──────────────────────────────┤
                                                                       ▼
                                                    lavSangLinje(sang)      (faelles.js)
                                                    "Titel  [Tilføj til køen]"
                                                                       │ klik
                                                                       ▼
                                                    tilfoejTilKoe(sang)     (koe.js)
                                                    koe.push(sang) → visKoe()
                                                                       │
                                                                       ▼
                                                    <div id="koe"> i højre kolonne
```

Rækkefølgen er vigtig: Opgave 1–4 (kø + hjælpefunktioner) bruges af alle de andre, så de laves først.

Opgave 1 – Kø-boksen (HTML) · fil: jukebox.html

> **Mål:** En boks i højre kolonne, hvor køen vises.

Udskift den tomme boks i højre kolonne med:

```
<div class="kolonne">
  <h3>Kø</h3>
  <div id="koe" style="overflow-y:auto">
  </div>
</div>
```

Hvorfor: `id="koe"` gør at JavaScript kan finde boksen med `getElementById("koe")` – præcis som vi gør med `"liste"`.

📖 w3schools – HTML id

Opgave 2 – Køen i JavaScript · fil: js/koe.js

> **Mål:** Gem køen i et array, og vis den i kø-boksen.

```
// Køen: et array UDEN FOR alle funktioner, så alle funktioner kan se det
let koe = [];

// Læg en sang i køen
function tilfoejTilKoe(sang) {
  koe.push(sang);   // tilføj bagerst i arrayet
  visKoe();         // tegn køen igen
}

// Tegn hele køen i boksen
function visKoe() {
  const koeBoks = document.getElementById("koe");
  koeBoks.innerHTML = "";               // tøm først
  for (const sang of koe) {
    const linje = document.createElement("div");
    linje.textContent = sang.title;
    koeBoks.appendChild(linje);
  }
}
```

Hvorfor står `let koe = []` udenfor funktionerne? En variabel lavet *inde i* en funktion forsvinder, når funktionen er færdig. Køen skal *huskes* mellem klikkene – derfor skal den være **global**.

Test: Skriv i konsollen (F12): `tilfoejTilKoe({ title: "Test" })` → "Test" skal dukke op i kø-boksen.

📖 w3schools – JS Scope · Array push()

Opgave 3 – Hjælpefunktion: hentJSON · fil: js/faelles.js

Mål: Skriv fetch + .json() ét sted i stedet for i hver funktion (DRY – "Don't Repeat Yourself").

```
// Henter en JSON-fil og giver indholdet tilbage
async function hentJSON(sti) {
  const svar = await fetch(sti);
  return await svar.json();
}
```

Brug den sådan i stedet for to linjer:

```
// Før:
const svarSange = await fetch("../db/sange.json");
const sange = await svarSange.json();

// Efter:
const sange = await hentJSON("../db/sange.json");
```

Det nye her er `return`: funktionen giver et resultat *tilbage* til den, der kaldte den.

📖 w3schools – JS Functions (return) · JS Async/Await

Opgave 4 – Hjælpefunktion: lavSangLinje · fil: js/faelles.js

Mål: Én funktion der laver "Sangtitel + [Tilføj til køen]". Den bruges både under Sange og under Albums.

```
// Laver én linje med sangtitel + "Tilføj til køen"-knap
function lavSangLinje(sang) {
  const linje = document.createElement("div");

  const titel = document.createElement("span");
  titel.textContent = sang.title + " ";

  const knap = document.createElement("button");
  knap.textContent = "Tilføj til køen";
  knap.className = "btn btn-sm btn-success";   // Bootstrap-knap (valgfrit)
  knap.addEventListener("click", function () {
    tilfoejTilKoe(sang);                       // ← forbindelsen til køen
  });

  linje.appendChild(titel);
  linje.appendChild(knap);
  return linje;            // giv linjen tilbage – kalderen bestemmer hvor den skal hen
}
```

Hvorfor `return` og ikke `appendChild` direkte? Så kan funktionen genbruges: under Sange sættes linjen i `#liste`, under Albums sættes den inde i et accordion-punkt.

📖 w3schools – addEventListener · createElement · Bootstrap 5 Buttons

Opgave 5 – Hjælpefunktion: lavAccordionPunkt · fil: js/faelles.js

> **Mål:** Én funktion der laver ét Bootstrap accordion-punkt (overskrift man kan klikke på + tom boks der folder ud).

Sådan ser ét accordion-punkt ud i ren HTML ifølge Bootstrap:

```
<div class="accordion-item">
  <h2 class="accordion-header">
    <button class="accordion-button collapsed" type="button"
            data-bs-toggle="collapse" data-bs-target="#album1">
      Back in Black
    </button>
  </h2>
  <div id="album1" class="accordion-collapse collapse">
    <div class="accordion-body">
      ← her kommer indholdet (sange eller albums)
    </div>
  </div>
</div>
```

- `data-bs-toggle="collapse"` = "Bootstrap, åbn/luk noget, når der klikkes her".
- `data-bs-target="#album1"` = "…nemlig det element, der har `id="album1"`".
- Derfor skal hvert punkt have sit **eget unikke id**, fx `"album" + album.album_id`.

Funktionen der bygger det:

```
// Laver ét accordion-punkt og giver det tilbage
function lavAccordionPunkt(id, titel) {
  const punkt = document.createElement("div");
  punkt.className = "accordion-item";
  punkt.innerHTML = `
    <h2 class="accordion-header">
      <button class="accordion-button collapsed" type="button"
              data-bs-toggle="collapse" data-bs-target="#${id}">
        ${titel}
      </button>
    </h2>
    <div id="${id}" class="accordion-collapse collapse">
      <div class="accordion-body"></div>
    </div>`;
  return punkt;
}
```

To nye ting:

- **Template literal** – tekst i `` `backticks` ``, hvor `${...}` sætter en variabel direkte ind i teksten.
- Vi bruger `innerHTML` her, fordi det ville kræve ca. 15 linjer `createElement` at bygge det samme.

📖 w3schools – Bootstrap 5 Collapse (inkl. Accordion) · JS Template Strings · innerHTML

Opgave 6 – Sange med "Tilføj til køen" · fil: js/sange.js

Mål: Hver sang i Sange-listen får sin egen "Tilføj til køen"-knap.

I løkken i `hentSange()` erstattes knappen med et kald til `lavSangLinje`:

```
for (const sang of sange) {
  liste.appendChild(lavSangLinje(sang));
}
```

Ekstra: Nu forsvinder kunstnernavnet. Hvordan får I det med igen? *Tip:* giv `lavSangLinje` en ekstra parameter: `lavSangLinje(sang, artistName)`.

Opgave 7 – Albums som accordion med sange · fil: js/albums.js

Mål: Hvert album er et accordion-punkt. Folder man det ud, ser man albummets sange – hver med en "Tilføj til køen"-knap.

`hentAlbums()` skal nu **også hente** `sange.json`. Løkken bliver:

```
for (const album of albums) {
  // 1. Lav selve accordion-punktet
  const punkt = lavAccordionPunkt("album" + album.album_id, album.title);

  // 2. Find den tomme boks inde i punktet
  const indhold = punkt.querySelector(".accordion-body");

  // 3. Find sangene der hører til DETTE album
  for (const sang of sange) {
    if (sang.album_id === album.album_id) {
      indhold.appendChild(lavSangLinje(sang));
    }
  }

  // 4. Sæt punktet ind i listen
  liste.appendChild(punkt);
}
```

Bemærk: Det er **samme for + if-mønster**, som I allerede bruger til at finde kunstnernavnet – bare med `album_id`.

Nyt: `querySelector(".accordion-body")` finder et element ud fra en CSS-klasse – ligesom `getElementById`, bare med en CSS-vælger. Kaldt på `punkt` leder den kun *inde i* punktet.

📖 w3schools – Element querySelector()

Opgave 8 – Kunstnere som accordion med albums · fil: js/artists.js

> **Mål:** Hver kunstner er et accordion-punkt. Folder man det ud, ser man kunstnerens albums.

`hentArtists()` skal nu **også hente** **`albums.json`**. Løkken bliver:

```
for (const kunstner of kunstnere) {
  const punkt = lavAccordionPunkt("kunstner" + kunstner.artist_id, kunstner.name);
  const indhold = punkt.querySelector(".accordion-body");

  for (const album of albums) {
    if (album.artist_id === kunstner.artist_id) {
      const linje = document.createElement("div");
      linje.textContent = album.title;
      indhold.appendChild(linje);
    }
  }

  liste.appendChild(punkt);
}
```

Bonus: Gør hvert album *inde i* kunstneren til sit eget accordion-punkt med sange (accordion i accordion = Opgave 7 + 8 kombineret). Husk unikke id'er, fx `"kunstnerAlbum" + album.album_id`.

Opgave 9 – Bonus (kun hvis der er tid)

Idé	Hint
Fjern en sang fra køen	En knap ved hver sang i `visKoe()`. 📖 Array splice()
"Afspil næste"	Fjern den første sang i køen og vis den som "Spiller nu". 📖 Array shift()
Antal i køen	Vis "Kø (3)" i overskriften. 📖 Array length
Limit på sange	Vis kun de første N sange. 📖 Array slice()

Del C – Fordeling, fejlfinding og links

Forslag til fordeling

Hvem	Opgaver	Filer
Én person (aftal hvem)	Del A – omlægning	alle
Shanjeef	1, 2, 3, 4, 5, 6 – kø + hjælpefunktioner + sange	jukebox.html, koe.js, faelles.js, sange.js
Abi & Phillip	7 – albums	albums.js
Simon & Oliver	8 – kunstnere	artists.js
Alle	9 – bonus	–

Hvordan kan Abi/Phillip og Simon/Oliver starte, før hjælpefunktionerne er færdige?
Byg først accordion-strukturen og vis sangtitlerne / albumtitlerne som almindelig tekst (`createElement("div")`). Når `lavSangLinje` er pushet, skiftes det ud med et kald til den.

Git-rytme (så vi undgår konflikter)

1. `git pull` – **altid** før man begynder at arbejde
2. Arbejd kun i *dine egne* filer
3. `git add .` → `git commit -m "kort beskrivelse"` → `git push`
4. Skal du ændre i en andens fil (fx `jukebox.html`), så sig det til gruppen først

Fejlfinding – åbn altid konsollen (F12 → Console)

Fejlbesked i konsollen	Hvad betyder det?	Løsning
`404 … js/albums.js`	Browseren kan ikke finde JS-filen	Tjek stavning og sti i `<script src="…">`
`404 … db/albums.json`	fetch kan ikke finde JSON-filen	Stien skal være `"../db/…"` (set fra HTML-filen)
`hentAlbums is not defined`	Funktionen findes ikke (endnu)	Er filen med i en `<script src>`? Er navnet stavet ens?
`lavSangLinje is not defined`	faelles.js er ikke indlæst	Tjek at `faelles.js` står **før** de andre filer
`CORS policy … file://`	Siden er åbnet med dobbeltklik	Åbn med **Live Server** i VS Code
Accordion folder ikke ud	Bootstrap JS mangler, eller id passer ikke	Tjek `bootstrap.bundle.min.js` og at `data-bs-target="#x"` matcher `id="x"`
Knapper er tomme	Forkert variabel, fx `sange.title`	Brug ental inde i løkken: `sang.title`

Ordliste

Begreb	Forklaring
Array	En liste af værdier, fx alle sange: `[ {...}, {...} ]`
Objekt	Én ting med egenskaber, fx én sang: `{ title: "...", album_id: 1 }`
Globalt scope	Variabler/funktioner uden for funktioner – kan ses af alle script-filer
`return`	Giver et resultat tilbage fra en funktion
`async` / `await`	"Vent på at noget langsomt (fx fetch) er færdigt, før du går videre"
DRY	Don't Repeat Yourself – skriv ting ét sted og genbrug dem
CDN	En server på nettet, hvor vi henter Bootstrap fra i stedet for at downloade det
Accordion	Bootstrap-komponent: overskrifter der folder indhold ud/ind

Samlet læseliste (w3schools)

Emne	Link
Eksterne JS-filer	w3schools.com/js/js_whereto.asp
Eksterne CSS-filer	w3schools.com/html/html_css.asp
Filstier	w3schools.com/html/html_filepaths.asp
Scope (global variabel)	w3schools.com/js/js_scope.asp
Funktioner og return	w3schools.com/js/js_functions.asp
Async / await	w3schools.com/js/js_async.asp
for...of	w3schools.com/js/js_loop_forof.asp
addEventListener	w3schools.com/js/js_htmldom_eventlistener.asp
Template strings	w3schools.com/js/js_string_templates.asp
querySelector	w3schools.com/jsref/met_element_queryselector.asp
Bootstrap 5 – kom i gang	w3schools.com/bootstrap5/bootstrap_get_started.php
Bootstrap 5 – Collapse/Accordion	w3schools.com/bootstrap5/bootstrap_collapse.php
Bootstrap 5 – Buttons	w3schools.com/bootstrap5/bootstrap_buttons.php